CS4433 – Software Engineering Final Project Report

Moderated Learning Management System (LMS)

Student	Katleho Senqhe
Student number	202101690
Email	katlehochristophersenqhe@gmail.com
Development model	Waterfall with V-Model verification and validation
Tools	C++17, CMake, SQLite, Catch2, Git, cppcheck
GitHub URL	https://github.com/katlehochristophersenqhe-maker/CS4433-LMS.git

Abstract

This report presents the complete software engineering life cycle for a Moderated Learning Management System developed for a secondary school. The system supports four roles—Administrator, Lecturer, Head of Department (HoD) and Student—and enforces a moderated assessment workflow in which marks remain hidden until the HoD releases them. Late submissions are accepted but incur a penalty of 10 % of the maximum mark for each whole day late, up to three days. Students may resubmit before the effective deadline; only the latest version is retained. Every mark change is recorded in an immutable audit log.

The project followed the Waterfall model with V-Model verification and validation. Phase 1 produced a feasibility study, work-breakdown structure, Gantt chart and risk register. Phase 2 resolved stakeholder conflicts, produced an SRS with 15 functional and 8 non-functional requirements, and three fully-dressed use cases. Phase 3 delivered a layered architecture, a class diagram applying the Strategy, Repository and Factory patterns, behavioural models and a Third-Normal-Form SQLite schema. Phase 4 implemented the system in C++17 with CMake, SQLite and a console UI. Phase 5 defined a V-Model test strategy with automated Catch2 tests and system/acceptance tests. Phase 6 established Git branching, continuous integration and static analysis. Phase 7 packaged the release. Phase 8 performed perfective maintenance for class-wide deadline extensions (v1.1) and addressed professional, legal and ethical issues under the Lesotho Data Protection Act 2011. The system meets all Must requirements with traceability from requirements through design, code and tests.

Table of Contents

1. Title page, abstract and table of contents	8. Configuration management and quality assurance
2. Introduction	9. Deployment
3. Project management plan	10. Maintenance (change request CR1)
4. Requirements engineering	11. Professional, legal and ethical issues
5. Software design document	12. Conclusion
6. Implementation	13. References
7. Verification and validation	14. Appendices

2. Introduction

Secondary schools need reliable tools for courses, assignments and assessment that preserve academic integrity. The Moderated LMS was developed under realistic constraints: offline operation on modest hardware, four user roles, late-penalty rules, HoD moderation before marks become visible, and a complete audit trail of every mark change. It is the individual final assignment for CS4433 at the National University of Lesotho and follows Waterfall with V-Model V&V; implementation began only after Gate 2. The system is a C++17 console application storing all data in a local SQLite database. Web or mobile interfaces, automated grading, plagiarism detection and online payment are deliberately excluded. This report shows how stakeholder conflicts were resolved, how the architecture and patterns support the business rules, how the implementation realises the design, how the system was verified, and how a post-release change request was handled. Requirements, design elements, source files and test cases are kept consistent, and the Git repository holds the code, tests, CI configuration, manuals and phase documents.

3. Project Management Plan

3.1 Feasibility study

Technical: high – the mandated tool chain is mature and free; the main risk (salted hashing and the multi-state submission life cycle) was mitigated by an early skeleton and CI. **Operational:** numbered menus keep cognitive load low for non-IT teachers. **Economic:** excellent – all tools are open source. **Legal:** compliance with the Lesotho Data Protection Act 2011 (minimal data, salted hashes, audit log). **Schedule:** acceptable within seven weeks if the Waterfall gates are respected, with buffer before each gate. Because school computers are old and the internet unreliable, the system is fully offline; SQLite needs no server and keeps the database in one file. Performance targets (login ≤ 5 s with 800 student records, assignment list ≤ 1.5 s) were written into the NFRs. **Recommendation: the project is feasible and should proceed.**

3.2 Work breakdown and schedule

WBS	Work package	Rationale / notes
1	Planning and management	Feasibility, schedule, initial risk register
2	Requirements engineering	Stakeholder analysis, conflict resolution, SRS, use cases
3	System design	Architecture, class diagram, patterns, behavioural models, database

4	Implementation	Largest package; C++17 + SQLite + design patterns
5	Verification and validation	Test plan, automated tests, system/acceptance tests, defect log
6	Configuration management & QA	Git discipline, CI, static analysis (continuous)
WBS	**Work package**	**Rationale / notes**
7	Deployment	Deployment diagram, installation guide, user manuals, v1.0 tag
8	Maintenance	CR1 impact analysis, v1.1 implementation, regression testing
9	Report, ethics, viva preparation	Runs throughout; final consolidation

A Gantt chart was prepared with dependencies, milestones (Gate 1 Requirements, Gate 2 Design, Gate 3 Testing) and a critical path through Requirements → Design → Implementation → Verification. One day of buffer was placed before each gate.

Figure 10 – Project Schedule (Gantt overview)

PNG:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/10_gantt.png draw.io
source:
https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/10_gantt.drawio

3.3 Risk management

A risk register of seven risks (technical, project, people) was maintained and reviewed at each phase gate. The highest-priority risks were password hashing, SQLite performance with 800 students, requirement ambiguity and insufficient test coverage.

ID	Risk	Category	Mitigation
R01	Difficulty implementing secure salted hashing	Technical	Use mature library / well-tested portable hash; prototype early
R02	SQLite performance with 800 students	Technical	3NF schema, indexes on FKs, early volume tests
R03	Ambiguous stakeholder requirements cause rework	Project	Conflict-resolution table; Gate 1 sign-off
R04	Inadequate test coverage (<70 %)	Project/Tech	Tests written alongside code; CI; coverage tools
R05	Illness, power cut or outage	People	Buffer day; offline-first Git; charged laptop
R06	Viva date compresses final preparation	People	Confirm date early; rehearse earlier if needed
R07	Late completion of Design (Gate 2)	Project	Strict Gantt adherence; buffer in Weeks 2–3

4. Requirements Engineering

4.1 Stakeholder analysis and conflict resolution

Six stakeholder groups were identified: Principal, Lecturer, HoD, Student, System Administrator and IT Officer. The Principal wants simplicity and HoD control of marks; the Lecturer wants rapid release and extensions; the HoD wants a late-penalty rule and moderation of every mark; the Student wants resubmission, written feedback and acceptance of late work; the Administrator needs account and enrolment management; the IT Officer needs offline operation, password protection and performance with 800 students. Three conflicts were resolved: (1) **mark visibility** – the Principal's rule was adopted: students see nothing until HoD release, though lecturers may mark and write feedback immediately; (2) **late work** – the HoD's 10 %-per-day rule (maximum three days, then zero) was adopted while still accepting the submission; (3) **resubmission** – unlimited before the effective deadline, only the latest version retained. Individual extensions are Must; class-wide extensions were added in Phase 8. Written feedback became a Must (FR-09, UC-02) and the IT Officer's constraints became measurable NFR-01, NFR-02 and NFR-07.

4.2 Software Requirements Specification

The SRS follows the spirit of ISO/IEC/IEEE 29148: 15 functional and 8 non-functional requirements, each uniquely identified, prioritised with

ID	Requirement (summary)	Priority	ID	Requirement (summary)	Priority
FR-01	Authenticate users; role-based access	Must	FR-08	Auto late penalty (10 %/day, max 3 days → 0)	Must
FR-02	Administrator manages user accounts	Must	FR-09	Lecturer marks: numeric mark + written feedback	Must
FR-03	Administrator manages departments, courses, enrolments	Must	FR-10	HoD moderates and releases marks	Must
FR-04	Lecturer creates assignment (title, deadline, max mark)	Must	FR-11	Student sees only released marks and feedback	Must
FR-05	Lecturer grants individual extension	Must	FR-13	Immutable audit log of every mark change	Must
FR-07	Student submits / resubmits before effective deadline	Must			

Key NFRs: login ≤ 5 s with 800 student records (NFR-01); fully offline (NFR-02); passwords stored only as salted hashes (NFR-03); mark change and audit entry in the same transaction (NFR-04); menu-driven UI for non-IT staff (NFR-05); graceful MoSCoW and given a verification method. Selected examples (full set in the repository; see Appendix A):

recovery from invalid input and database errors (NFR-06); C++17, CMake and a documented coding standard (NFR-08).

4.3 Use-case model

A use-case diagram covers the four actors with include, extend and generalisation relationships. Three use cases are fully dressed. **UC-01 Submit Assignment** covers on-time, late (automatic penalty, BR-01) and resubmission (BR-02, latest only) paths. **UC-02 Mark Submission** requires a numeric mark and written feedback and leaves the mark invisible to the student. **UC-03 Moderate and Release Marks** requires a reason for any change and records it in the audit log before the status becomes Released.

5. Software Design Document

5.1 Architecture

A layered architecture with four layers was selected: Presentation (console menus), Application/Business Logic (services enforcing permissions and rules), Domain (entities and pure rules) and Data Access (repository interfaces and SQLite implementations). Dependencies point strictly downward, giving business rules a clear home, supporting transactional audit logging and keeping SQL out of the presentation layer. Model–View–Controller was considered and rejected: it suits rich graphical interfaces, whereas the LMS needs only a simple console UI.

Figure 1 – Layered Architecture

PNG:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/01_architecture.png
draw.io source:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/01_architecture.drawio
Principal components: ConsoleUI, AuthService, AdminService, LecturerService, HoDService, StudentService, the repository interfaces and SQLite implementations, LatePenaltyStrategy and UserFactory. All database access uses parameterised queries.

Figure 2 – Component Diagram

PNG:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/02_component.png draw.io
source:
https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/02_component.drawio

5.2 Class diagram and design patterns

An abstract *User* class holds common attributes (identifier, username, password hash, full name, active flag); Student, Lecturer, HeadOfDepartment and Administrator provide role-specific behaviour, and services use polymorphism. **Strategy:** LatePenaltyStrategy / StandardLatePenaltyStrategy implement the school rule, so a new policy needs no change to the submission service (Open/Closed). **Repository/DAO:** one interface per aggregate with SQLite implementations; services depend only on abstractions (Dependency Inversion). **Factory:** UserFactory centralises creation of the correct User subclass, removing switch statements. The design follows SOLID: single responsibility, open/closed penalty strategy, substitutable User subclasses, narrow repository interfaces and dependence on abstractions. Multiplicities: Department 1..* Course; Course 1..* Assignment and Enrolment; Student 1..* Enrolment and Submission; Assignment 1..* Submission and 0..* Extension; Submission 0..* AuditLogEntry; Lecturer linked to Courses; HoD linked to one Department. These match the foreign keys in the schema. Data members are private/protected, the public contract is public and helpers are private.

Figure 3 – Class Diagram (core)

PNG:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/03_class_diagram.png draw.io
source:
https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/03_class_diagram.drawio

5.3 Behavioural models

The Submission state machine runs Submitted → Late → Marked → Moderated → Released; resubmission is allowed only while Submitted or Late and before the effective deadline, and marks stay invisible until Released.

Figure 4 – Submission State Machine

PNG:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/04_state_machine.png
draw.io source:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/04_state_machine.drawio

The swimlane activity diagram (Student, System, Lecturer, HoD) shows the process from submission to released mark, including the moderation loop and the rule that every mark change carries a reason and an audit entry.

Figure 5 – Activity Diagram (swimlanes)

PNG:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/05_activity.png
draw.io source:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/05_activity.drawio
Sequence diagrams for UC-01 and UC-03 show the interactions among ConsoleUI, services, repositories, the penalty strategy and the database, including the late, resubmission and error paths.

Figure 6 – Sequence Diagram: Submit Assignment (UC-01)

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/06_sequence_submit.png draw.io
source: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/06_sequence_submit.drawio

Figure 7 – Sequence Diagram: Moderate and Release (UC-03)

PNG:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/07_sequence_moderate.png draw.io
source:
https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/07_sequence_moderate.drawio

5.4 Database design

The logical model is in Third Normal Form: User (single-table inheritance via a CHECK-constrained role column), Department, Course, Enrolment, Assignment, Extension (nullable student_id permits class-wide extensions), Submission (UNIQUE assignment_id + student_id, so resubmission replaces the previous row – BR-02) and Audit_Log (append-only, reason NOT NULL). final_mark is stored on Submission for query performance while the full history resides in Audit_Log. Single-table inheritance suits authentication (one lookup, few role-specific attributes). Password hash and salt never leave the data-access layer; indexes on foreign keys and on status/department_id support the 800-student target.

Figure 8 – Entity-Relationship Diagram (3NF)

PNG:https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/08_erd.png draw.io
source:
https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/08_erd.drawio

6. Implementation

The source tree mirrors the architecture (include/lms/{domain,app,infra,ui} with matching src directories). Domain classes contain no SQL; services orchestrate use cases and enforce permissions; infrastructure implements repositories with parameterised SQLite statements; ConsoleUI only displays menus and forwards requests. One CMakeLists.txt builds the libraries, the executable and the Catch2 tests; the

SQLite amalgamation is compiled as C and linked statically, so the binary is self-contained and offline. A documented coding standard (naming, formatting, exceptions) is followed. The abstract User, UserFactory and StandardLatePenaltyStrategy are implemented as designed, and mark changes and audit inserts share one transaction (NFR-04).

Security and persistence: plain-text passwords are never stored; PasswordHasher produces a per-user-salted SHA-256 hash (the interface is ready for Argon2) and verification recomputes and compares it. All SQL uses parameter binding. Input validation and exceptions stop invalid data or database errors from terminating the process (NFR-06). **Business rules** implemented and tested: late penalty, individual and class-wide extensions, latest-only resubmission, mandatory written feedback, moderation with reason, release, immutable audit log and role-based access at the service boundary. Menus show only role-permitted operations, and empty content, out-of-range marks and missing reasons are rejected before any write. Six demonstration accounts covering all roles, with sample assignments in Submitted, Late, Marked and Released states, are seeded on first run.

7. Verification and Validation

The V-Model strategy pairs each artefact with a test level: **unit** tests (late-penalty strategy, password hasher, user factory); **integration** tests (services with SQLite repositories: authentication, user creation, secure storage); **system** tests from the use cases (ST-01 submit/late/resubmit; ST-02 marking with feedback and hidden mark; ST-03 moderation and release); and **acceptance** tests for the Must requirements. Entry criteria are a successful build and green lower levels; exit criteria are no open critical defects and the coverage target. Equivalence partitioning and boundary-value analysis were applied to days late (0, 1, 3, 4 → no penalty, 10 %, 30 %, zero), mark ranges (0, maxMark) and passwords (correct, incorrect, unknown user, empty). Security tests confirm the hash column holds no plain text and that cross-role calls (e.g. a Student calling a Lecturer method) are rejected before any side effect.

Results: Catch2 reports **65 assertions across 25 test cases, all passing**, including the CR1 tests. Line coverage of the business-logic layer (domain + application services) meets the 70 % target, measured with compiler instrumentation and lcov; the presentation layer and SQLite amalgamation are excluded. Exception flows (empty content, out-of-range mark, missing reason) give friendly messages without crashing. The defect log (severity, cause, fix, retest) has no open critical defects.

8. Configuration Management and Quality Assurance

Git is used with feature branches merged by pull request into main, meaningful commit messages and tagged releases (v1.0; v1.1 for CR1). A GitHub Actions workflow builds the project and runs the Catch2 suite on every push. cppcheck (optionally clang-tidy) is run on the source tree and findings are fixed or justified; code metrics (lines of code, cyclomatic complexity of key functions) are recorded; and a self-review checklist (naming, exception safety, parameterised SQL, no magic numbers, design consistency) was applied, with changes noted in the repository history.

9. Deployment

The UML deployment diagram shows one school workstation (or small network share) hosting the executable and the SQLite database file; no application server or external network service is required.

Figure 9 – UML Deployment Diagram

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/09_deployment.png
draw.io source: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/09_deployment.drawio

The installation guide for a school technician covers installing a C++17 toolchain (or unpacking a pre-built archive), placing the executable and schema in a writable directory, running once to create the database and optionally loading the demo accounts. Role-specific manuals (Administrator, Lecturer, HoD, Student) list every menu option, input and business rule. Release notes for v1.0 summarise the implemented Must requirements and known limitations (console-only UI, single-file deployment).

10. Maintenance (Change Request CR1)

Classification and impact analysis. CR1: “A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student’s assignment list.” It is **perfective** maintenance: it meets an additional user need and neither corrects a defect nor adapts to a changed environment. Affected artefacts: the extension requirements and business rules, Lecturer and Student use cases, class diagram (new grantClassExtension), sequence and activity diagrams, LecturerService, StudentService, ConsoleUI, the Extension repository (already NULL-capable), the student assignment list and the tests. No migration is needed for fresh installs; a transactional migration is provided for legacy databases whose student_id was NOT NULL.

Implementation (v1.1). LecturerService::grantClassExtension validates the request and stores an Extension row with null student_id. The student list shows the effective deadline (the later of the original deadline and any applicable extension; individual extensions take precedence), and the lecturer menu gains “Grant class extension”. **Regression testing.** test_cr1_extensions.cpp checks class-wide persistence, effective-deadline visibility, correct late penalty with a class extension, precedence rules and validation failures. The full suite (65 assertions, 25 test cases) passes and the defect log records no regressions.

11. Professional, Legal and Ethical Issues

The system processes learners’ names, identifiers and assessment records. The Lesotho Data Protection Act 2011 requires lawful, fair and transparent processing, adequate and non-excessive data, and technical protection against unauthorised access. The design stores only assessment data, never stores plain-text passwords, restricts each role to the operations it needs, requires a non-empty reason for any moderation change and keeps an immutable audit log that supports accountability if a mark is disputed. **Accessibility** is limited by the console interface; a graphical or screen-reader-friendly front-end is future work. **Fairness:** the uniform 10 %-per-day penalty treats all students equally after the deadline while individual and class-wide extensions cater for documented need, so the rule is transparent and consistently applied. Professionally, the project used a documented life cycle, version control, automated testing and honest defect reporting; claims are limited to what the tests exercise.

12. Conclusion

The Moderated LMS went through the full CS4433 life cycle. Conflicts were resolved in favour of academic integrity (HoD moderation before release) while accommodating late work, resubmission and written feedback. A layered architecture, three classical patterns and a normalised SQLite schema map cleanly onto the C++17 implementation, and automated tests, CI and static analysis provide evidence of quality. The V-Model caught defects close to their origin (penalty arithmetic and hasher errors at unit level, an audit-log transaction issue at integration level), and CR1 was analysed, implemented and regression-tested without disturbing existing behaviour. Limitations: console-only UI and single-workstation deployment. Future work: a lightweight graphical shell, richer HoD reporting and Argon2 hashing. Within scope, the system satisfies all Must requirements and is ready for supervised use.

13. References

Boehm, B.W. (1981) *Software engineering economics*. Englewood Cliffs, NJ: Prentice-Hall.

Boehm, B.W. (1991) 'Software risk management: principles and practices', *IEEE Software*, 8(1), pp. 32–41.

Cockburn, A. (2001) *Writing effective use cases*. Boston: Addison-Wesley.

Fowler, M. (2002) *Patterns of enterprise application architecture*. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1995) *Design patterns: elements of reusable object-oriented software*. Reading, MA: Addison-Wesley.

Government of Lesotho (2011) *Data Protection Act, 2011 (Act No. 5 of 2011)*. Maseru: Government Printer.

IEEE (2011) *ISO/IEC/IEEE 29148:2011 Systems and software engineering — Life cycle processes — Requirements engineering*. New York: IEEE.

Martin, R.C. (2017) *Clean architecture: a craftsman's guide to software structure and design*. Boston: Prentice Hall.

Pressman, R.S. and Maxim, B.R. (2015) *Software engineering: a practitioner's approach*. 8th edn. New York: McGraw-Hill Education.

Project Management Institute (2021) *A guide to the project management body of knowledge (PMBOK guide)*. 7th edn. Newtown Square, PA: PMI.

Sommerville, I. (2016) *Software engineering*. 10th edn. Harlow: Pearson.

Wiegers, K. and Beatty, J. (2013) *Software requirements*. 3rd edn. Redmond: Microsoft Press.

Catch2 documentation. Available at: https://github.com/catchorg/Catch2 (Accessed: November 2026).

14. Appendices

Appendix A – Requirements Traceability Matrix (excerpt; full matrix in the repository)

FR	Use case	Design / class	Source / test
FR-01	Login	AuthService, User hierarchy	AuthService.cpp; IT-AUTH-*
FR-07	UC-01	StudentService, Submission	StudentService.cpp; ST-01
FR-08	UC-01 (late)	LatePenaltyStrategy	LatePenaltyStrategy.cpp; UT-LP-*
FR-09	UC-02	LecturerService, AuditLog	LecturerService.cpp; ST-02
FR-10	UC-03	HoDService, state machine	HoDService.cpp; ST-03
FR-13	UC-02/03	Audit_Log table, repository	SqliteAuditRepository; audit tests

Appendix B – Test results summary: Catch2 run (post-CR1): 25 test cases, 65 assertions, 0 failures. Coverage of domain + application services ≥ 70 %. System tests ST-01, ST-02 and ST-03 passed, including exception flows. No open critical or major defects. **Appendix C – User manuals (summary):** role-specific manuals are in docs/USER_MANUALS.md and list every menu option and business rule; demonstration accounts (admin, lec1, hod1, stu1, …) are seeded on first run. **Appendix D – Updated risk register:** all seven risks were reviewed at each gate; R01 (hashing) and R04 (coverage) fell to low residual risk after the security prototype and automated suite, R03 closed after Gate 1, and R05 and R06 remain monitored but have not materialised.

15. List of Figures (GitHub hyperlinks)

All diagrams are stored in the repository under docs/diagrams/ as a PNG export and a draw.io source file.

Figure 1 – Layered Architecture

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/01_architecture.png
draw.io: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/01_architecture.drawio

Figure 2 – Component Diagram

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/02_component.png
draw.io: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/02_component.drawio

Figure 3 – Class Diagram (core)

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/03_class_diagram.png
draw.io: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/03_class_diagram.drawio

Figure 4 – Submission State Machine

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/04_state_machine.png
draw.io: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/04_state_machine.drawio

Figure 5 – Activity Diagram (swimlanes)

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/05_activity.png
draw.io: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/05_activity.drawio

Figure 6 – Sequence Diagram: Submit Assignment (UC-01)

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/06_sequence_submit.png
draw.io: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/06_sequence_submit.drawio

Figure 7 – Sequence Diagram: Moderate and Release (UC-03)

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/07_sequence_moderate.png
draw.io: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/07_sequence_moderate.drawio

igure 8 – Entity-Relationship Diagram (3NF)

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/08_erd.png
draw.io: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/08_erd.drawio

Figure 9 – UML Deployment Diagram

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/09_deployment.png
draw.io: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/09_deployment.drawio

Figure 10 – Project Schedule (Gantt overview)

PNG: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/10_gantt.png
draw.io: https://github.com/katlehochristophersenqhe-maker/CS4433-LMS/blob/main/github_assets/docs/diagrams/10_gantt.drawio